Infosys Springboard Virtual Internship 7.0 Completion Report

Team Details: Team - A
Ankit Patel
Ashwin Sai
Divyashree M
Esakki Saravanan
Godugula Monika

Batch Number: 1
Start date : 29-06-2026

Names:

1	Ankit Patel
2	Ashwin Sai
3	Divyashree M
4	Esakki Saravanan
5	Godugula Monika

Internship Duration: 8 Weeks

1. Project Title

Intelligent Software Defect Tracking System With Resolution Assistance

2. Project Objective

The objective of this project is to build an **Intelligent Software Defect Tracking System with Resolution Assistance** that helps software teams efficiently manage and analyze defects by:

- Tracking and managing software defects throughout their life cycle using details such as Bug ID, Module, Root Cause, Date Reported, and Date Closed.

- Analyzing defect patterns to identify modules generating the highest number of bugs and root causes contributing most frequently to software defects.
- Calculating and comparing defect resolution time using Date Reported and Date Closed to identify delayed defects and modules or root causes requiring more time for resolution.
- Providing visual insights through interactive charts, summaries, KPIs, and trend trend analysis to make defect information easier to understand and support decision-making.
- Enabling efficient defect analysis through filtering, searching, and detailed bug records, helping teams prioritize important and recurring issues.

3. Project description in detail

Intelligent Software Defect Tracking System With Resolution Assistance is a software defect management and analysis system designed to help development and testing teams record, monitor, analyze, and resolve software defects in a structured manner. In software development, a large number of defects may occur across different modules due to various root causes, making it difficult to manually track defect patterns and identify areas that require immediate attention. This system addresses these challenges by organizing defect information and converting it into meaningful analytical insights.

The system allows users to maintain software defect records using important details such as **Bug ID, Module, Root Cause, Date Reported, and Date Closed**. The collected defect data is processed to understand the distribution of bugs across different modules and identify the root causes responsible for a higher number of defects. This helps teams recognize frequently affected modules and recurring defect causes that may require additional testing, debugging, or corrective measures.

The system provides an **interactive dashboard** containing charts, summaries, KPIs, and detailed defect records. Module-wise bug counts, root-cause distribution, resolution-time comparisons, and defect trends are presented through visualizations, making complex defect information easier to understand. Users can also apply filters and search through defect records to focus on specific modules, root causes, or individual bugs.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Requirement analysis and planning	Defined features, system architecture, and defect attributes: Bug ID, Root Cause
Week 2	Frontend design	Built defect entry form, Dashboard, and filters using Streamlit
Week 3	Backend development	Implemented pandas for data processing, cleaning and resolution time calculation
Week 4	Database integration	Defect data persistence implemented using CSV/excel with pandas
Week 5	Dashboard and analytics	Module-wise defects, Root Cause charts, and KPIs added using plotly
Week 6	AI Insights	Rule-based Resolution Assistance implemented to suggest fixes for high-risk modules
Week 7	Security and reports	Added login, data validation, and CSV/PDF export for defect reports
Week 8	Testing and documentation	System tested with 200+ records, bug fixes, and final report + screenshots prepared

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Defined objectives, gathered requirements, finalized scope, and planned workflow for defect tracking and resolution assistance system.	29 – Jun - 2026
Prototype / First Draft	Developed initial version with defect entry module, basic KPIs, and Streamlit dashboard to validate core functionality.	10 - Jul - 2026
Mid-Term Review	Demonstrated working dashboard with defect data processing, KPIs, and interactive charts; received feedback and analysis logic.	24 - Jul - 2026
Final Submission	Completed full functionality including defect tracking, resolution assistance, root cause analysis, dashboards, and secure deployment; project deployed for evaluation.	06 - Aug – 2026
Presentation -	Presented the complete working system with live demonstration of defect tracking, resolution assistance features, dashboards, analytics and overall project workflow.	18- Aug - 2026

5b. Project execution details

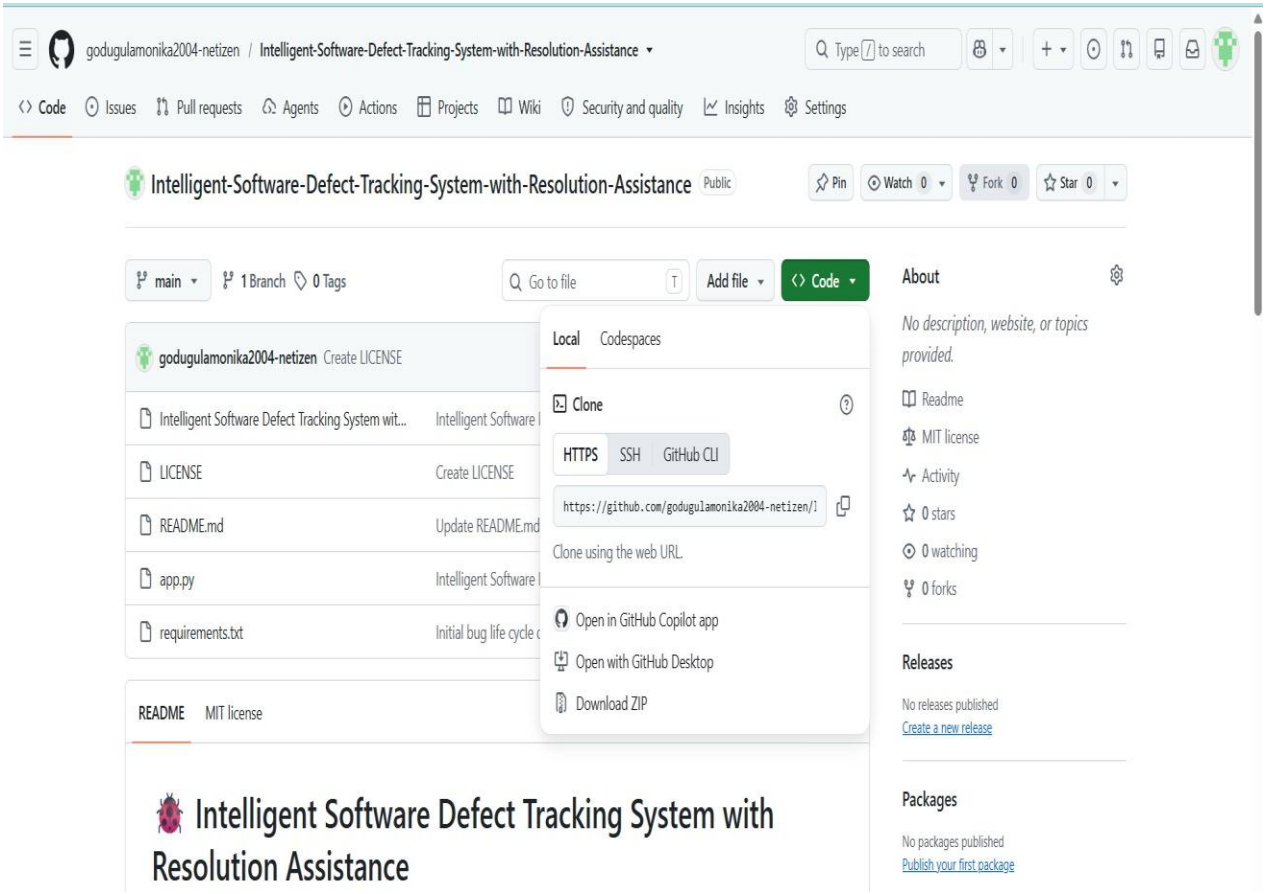
- **Requirement Analysis:** Identified the need for an organized software defect tracking and analysis system and defined the requirements for monitoring Bug ID, Module, Root Cause, Date Reported, Date Closed, and resolution time.
- **Technology Selection:** Selected **Python** as the primary programming language, **Pandas** for data processing and analysis, **Streamlit** for developing the interactive dashboard, and **Plotly** for creating charts and visualizations.
- **System Design:** Planned the overall workflow of the defect tracking system, including data loading, preprocessing, defect analysis, resolution-time calculation, visualization, filtering, and reporting.
- **Defect Data Management:** Imported the software defect dataset and organized important fields such as Bug ID, Module, Root Cause, Date Reported, and Date Closed. The data was cleaned and prepared for further analysis.
- **Dashboard Development:** Developed an interactive **Streamlit dashboard** containing KPI cards, module-wise bug charts, root-cause distributions, resolution-time analysis, defect trends, filters, and detailed bug records.
- **Data Visualization:** Created interactive visualizations using **Plotly**, including bar charts, pie charts, and trend charts to present defect patterns, root causes, module-wise bugs, and resolution performance in an easy-to-understand format.
- **Filtering & Search Implementation:** Added filtering options for relevant defect categories and search functionality to allow users to examine specific modules, root causes, or bug records.
- **Version Control & Deployment:** Used **Git and GitHub** to maintain and manage the project source code. The Streamlit application was deployed through **Streamlit Community Cloud** to make the dashboard accessible through a web browser.
- **Documentation:** Prepared the project documentation, including the project objectives, description, technologies used, execution details, screenshots, and results.

6. Snapshots / Screenshots

Step 1: Upload the Project to GitHub

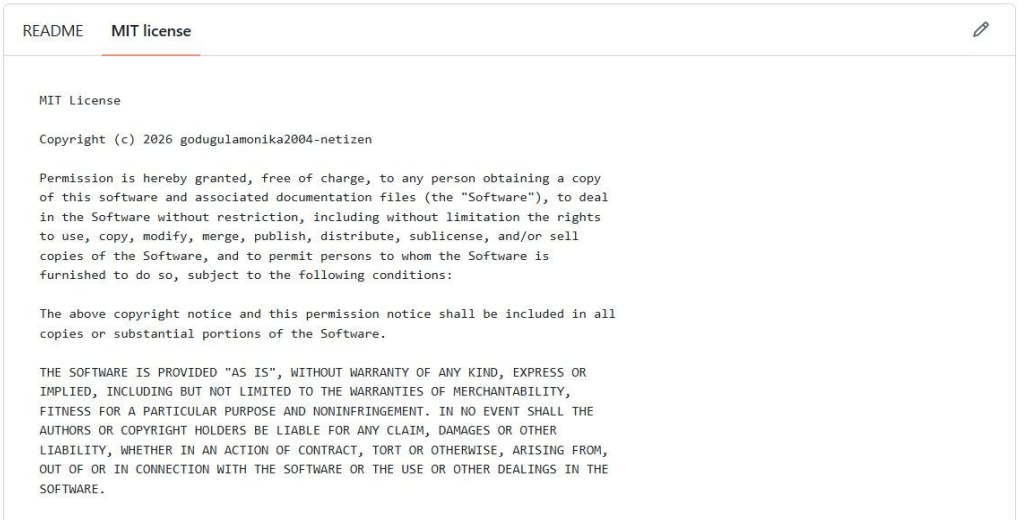
The project was uploaded to the GitHub repository using **Git Bash**. The project files were added, committed, and pushed to GitHub. After uploading, the repository was opened in **VS Code** for further execution and testing.

GitHub Repository:
https://github.com/godugulamonika2004-netizen/Intelligent-Software-Defect-Tracking-Systemwith-Resolution-Assistance.git



Step 2: Add MIT License

An **MIT License** was added to the GitHub repository to define the permissions for using, modifying, copying, and distributing the project. The license allows others to use the source code with proper attribution and subject to the MIT License conditions.


README MIT license

MIT License

Copyright (c) 2026 godugulamonika2004-netizen

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Step3 : Dashboard

The dashboard provides a **centralized and interactive platform for software defect tracking**. It combines filtering, KPI reporting, visual analysis, bug identification, duplicate detection, and resolution assistance. This enables project teams to monitor defect trends, prioritize critical bugs, track resolution performance, and make better decisions during software development.

Key Performance Indicators (KPIs)

The **Key Performance Indicators** section summarizes the current defect status.

- **Total Bugs – 200:** Indicates that 200 bug records are available in the selected dataset/filter.
- **Closed Bugs – 33:** Shows that 33 bugs have been successfully closed.
- **Average Resolution Time – 25.48 hours:** Represents the average time required to resolve bugs.
- **Defect Density – 10.00 bugs/module:** Indicates the concentration of defects relative to the software modules being analyzed.

Dashboard Visualization:

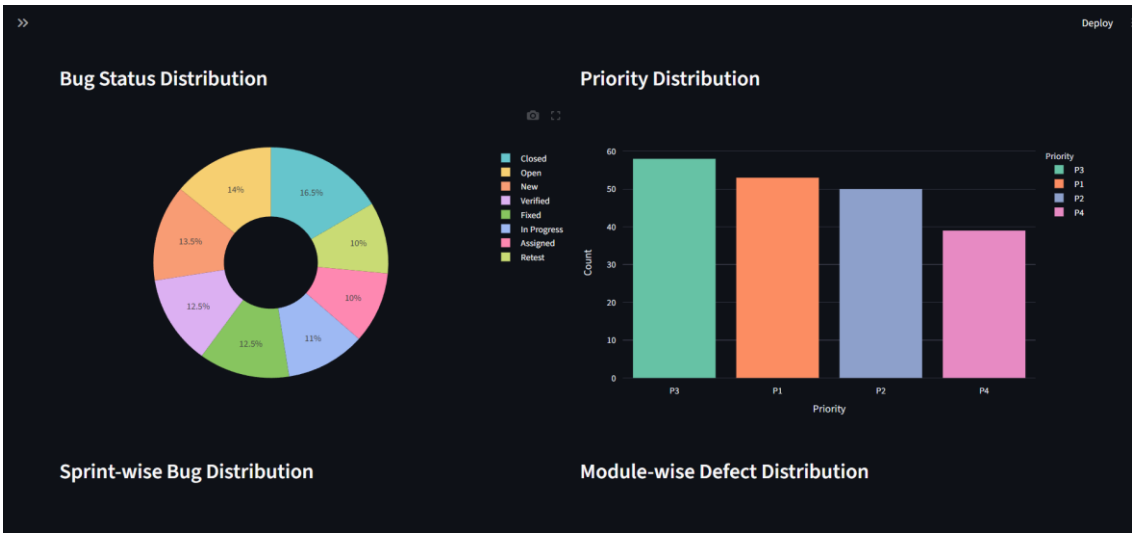
The dashboard provides a graphical representation of software defects and their distribution. It helps users monitor the current status of bugs, analyze their priority, and understand defect patterns across sprints and software modules.

Bug Status Distribution:

The donut chart represents the percentage of bugs according to their current status. Closed bugs account for 16.5%, Open bugs 14%, New bugs 13.5%, Verified bugs 12.5%, Fixed bugs 12.5%, In Progress bugs 11%, Assigned bugs 10%, and Retest bugs 10%.

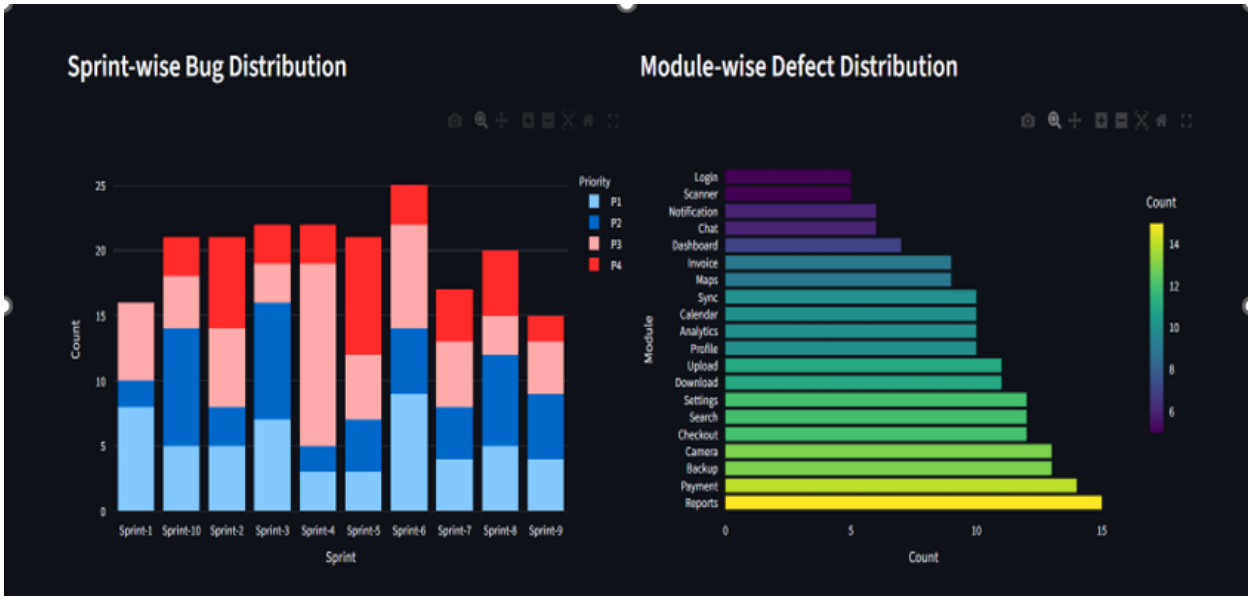
Priority Distribution:

The bar chart shows the number of bugs according to priority. P3 has 58 bugs, P1 has 53 bugs, P2 has 50 bugs, and P4 has 39 bugs. This helps identify the priority levels with the highest number of defects.

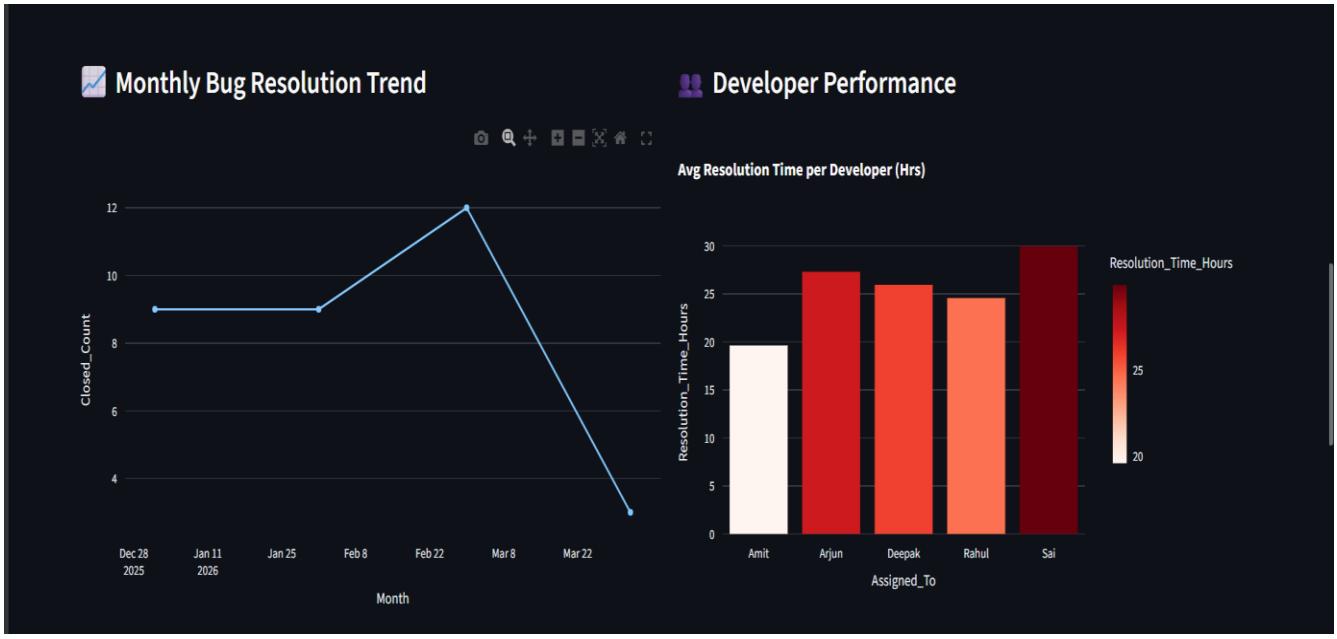


Sprint-wise Bug Distribution:
This section is used to analyze and compare the number of bugs reported in different sprints. It helps identify sprints with higher defect counts and supports monitoring of defect trends during development.

Module-wise Defect Distribution:
This section represents the distribution of defects across different software modules. It helps identify modules that contain a higher number of defects and may require additional testing or maintenance.



Monthly Bug Resolution and Developer Performance:
The image shows two important analytical components of the Intelligent Software Defect Tracking System: the Monthly Bug Resolution Trend and Developer Performance. These charts help evaluate bug resolution progress and developer-wise resolution efficiency.



AI Resolution Assistance:
The images show the AI Resolution Assistance module of the Intelligent Software Defect Tracking System. This module analyzes the details of a reported software defect and provides an intelligent suggestion for identifying the cause, resolving the issue, and verifying the fix.

Bug Information
The Bug Information section is used to enter the details of the defect.

- **Bug Title:** *Payment transaction failed*
- **Bug Description:** The customer completes the payment, but the order remains in a pending status.
- **Affected Module:** *Checkout*

After entering the information, the user clicks the **Analyze Defect** button to start the AI-based analysis.

7. Challenges Faced

Data Preprocessing and Cleaning :
Handling the bug dataset required careful preprocessing to ensure that missing values, inconsistent formats, duplicate records, and incorrect entries did not affect the analysis. The data had to be cleaned and structured properly before generating meaningful insights.

Date Format and Resolution Time Calculation :
Calculating the resolution time from **Date_Reported** and **Date_Closed** was challenging because date values were not always in the same format. Proper date conversion and validation were required to accurately calculate the number of hours taken to resolve each defect.

Bug Classification and Analysis :
Analyzing defects based on **Module, Priority, Status, and Root Cause** required careful classification. Identifying which modules and root causes contributed to the highest number of bugs was important for generating useful project insights.

Dashboard Visualization :
Developing an interactive dashboard using **Python, Pandas, Streamlit, and Plotly** involved challenges in designing clear visualizations and presenting large amounts of bug information in an understandable format. Charts and metrics had to be arranged properly for effective monitoring.

Module-Wise and Root-Cause Analysis :
Generating module-wise and root-cause distributions required accurate aggregation of the dataset. Ensuring that the charts correctly represented the number of defects and their contributing causes required repeated testing and validation.

Resolution Time Analysis :
Analyzing average resolution time across different modules, priorities, and defect categories was challenging. The calculations needed to handle missing or invalid dates while providing reliable information about defect resolution performance.

Interactive Filtering and User Experience :
Implementing filters for parameters such as Module, Priority, Status, and Root Cause required careful handling of the dataset and dashboard state. The dashboard needed to update all related metrics and visualizations correctly whenever users changed the filters.

Dashboard Performance and Data Handling :
Loading and processing the bug dataset efficiently was important to maintain a responsive dashboard. Data caching and optimized Pandas operations were used to reduce unnecessary processing and improve the overall dashboard performance.

Presentation and Documentation Preparation :
Preparing screenshots, explaining the project workflow, documenting the dashboard features, and aligning the project report with the Infosys Springboard internship format required careful planning. The final documentation had to clearly explain the implementation, analysis, results, and project outcomes.

8. Learnings & Skills Acquired

- **Python Programming & Data Analysis**

Gained practical experience in using Python and Pandas for loading, cleaning, transforming, and analyzing software defect data. Learned how to work with structured datasets and extract meaningful information from bug records.

- **Streamlit Dashboard Development**

Developed practical knowledge of building interactive dashboards using Streamlit. Learned how to create dashboard layouts, display key metrics, implement interactive components, and present defect-related information in an easy-to-understand format.

- **Data Visualization & Analytics**

Improved skills in creating interactive charts using Plotly to visualize bug distribution, module-wise defects, root-cause analysis, priority levels, status distribution, and resolution-time trends.

● **Bug Lifecycle Analysis**
Gained a better understanding of the software defect lifecycle, including defect reporting, classification, prioritization, assignment, resolution, and closure. Learned how to analyze defect patterns to support better software quality management.

● **Data Cleaning & Preprocessing**
Learned techniques for handling missing values, duplicate records, inconsistent data formats, and invalid entries. Improved understanding of preparing reliable datasets before performing analysis and visualization.

● **Date Handling & Resolution Time Analysis**
Developed skills in converting and validating date fields such as Date_Reported and Date_Closed. Learned how to calculate Resolution_Time_Hours and use the results to evaluate defect resolution performance.

● **Module-Wise & Root-Cause Analysis**
Learned how to analyze defects based on Module and Root Cause to identify areas generating the highest number of bugs. This improved analytical thinking and helped in identifying potential areas for software improvement.

● **Interactive Filtering & Dashboard Design**
Gained experience in implementing filters based on Module, Priority, Status, and Root Cause. Learned how interactive filtering can help users explore specific defect categories and obtain focused insights.

● **Problem Solving & Debugging**
Strengthened troubleshooting skills by resolving issues related to Python libraries, CSV filepaths,date formatting, Pandas processing, Streamlit execution, and dashboard errors. Learned to identify errors systematically and implement suitable solutions.

● **Git & GitHub Skills**
Gained practical experience using Git and GitHub for version control, repository management, committing project files, managing branches, and pushing the project code to a remote repository.

- **Deployment & Environment Management**

Learned the basic process of deploying a Streamlit application and managing project dependencies using a requirements file. Gained experience in configuring the development environment and ensuring that the application runs correctly.

- **Project Planning & Execution**

Improved project management skills by dividing the project into phases such as data collection, preprocessing, analysis, dashboard development, visualization, testing, deployment, and documentation.

- **Presentation & Documentation Skills**

Improved the ability to prepare structured project documentation, explain the system workflow, describe dashboard functionality, present analytical results, and demonstrate the project effectively during Infosys Springboard milestone reviews.

Conclusion :

The Intelligent Software Defect Tracking System with Resolution Assistance provides an organized approach to managing and analyzing software defects. The system transforms raw defect records into meaningful information by analyzing modules, root causes, defect trends, and resolution times. The interactive Streamlit dashboard makes the analysis easier to understand and allows users to explore defect information through filters, charts, KPIs, and detailed records. By identifying frequently affected modules, common root causes, and defects with longer resolution times, the system can assist software teams in focusing their efforts on important quality issues and improving the overall efficiency of defect management.

Acknowledgements

This section acknowledges the organization, mentor, and team members who supported and guided me throughout my internship journey.

I would like to express my sincere gratitude to Infosys for providing me the Valuable opportunity to work on the Intelligent Software Defect Tracking System with Resolution Assistance.

This internship has significantly enhanced my technical Knowledge and Real-world problem-solving skills.

I am especially thankful to my mentor, Vamsi Mitra, for his constant guidance, support, and insightful feedback throughout the development of this project. His mentorship helped me understand project architecture, improve implementation strategies, and overcome technical challenges effectively.

I would also like to thank my team members for their collaboration, encouragement, and contribution during different phases of the project. Working together strengthened our coordination skills and made the overall experience productive and enriching.

Finally, I extend my appreciation to everyone who supported me directly or indirectly during this internship journey and contributed to the successful completion of this project.